

p3_report(6)

April 10, 2026

Author: Yuze Fu

The Goal of this project is to investigate the properties of SBP-SAT schemes and compare them to the schemes with ghost cells and one-sided difference formula. We will test the sixth order Runge-Kutta scheme with SBP(2, 1), SBP(4, 2), SBP(6, 3) and SBP(4, 4).

1 Design

1.1 Choice of PDE

I use the following PDE for testing:

$$\begin{cases} u_t(x, t) + cu_x(x, t) = 0, & a \leq x \leq b, 0 \leq t \leq T, \\ u(x, 0) = f(x), & a \leq x \leq b, \\ u(a, t) = g(t), & 0 \leq t \leq T. \end{cases}$$

whose solution is $u(x, t) = f(x - ct)$ if f is smooth. In this project, I fix

$$u(x, t) = \exp(-(x - ct)^2) + \cos(13(x - ct))$$

with

$$c = 1.1, a = -1 \text{ and } b = 1$$

1.2 Verification Tests for SBP-SAT Operators

First, We want to test D, H, H^{-1} imported from sbp.py satisfy equation (1) or not.

$$HD + D^*H = B \tag{1}$$

Then, we want to test the order of the SBP operator D by using equation (2) where x is the grids.

$$D(x^k) = kx^{k-1} \tag{2}$$

We will test from $k = 0$ and increase one each time. Assume it breaks at $k = p$, and p is just the order.

For both verifications, we'd like to test them with different spatial steps m . To compare the matrices and vectors, we choose to use l_2 -norm and set the tolerance to be 10^{-10} .

1.3 Convergence and Stability Tests

To test the order of convergence and stability, the main variable we care about is m which is the number of nodes between a and b . Thus, I will fix the other variables

$$CFL = 0.5 \text{ and } T = 1$$

We know that the theoretical stability condition is $CFL \leq 1$. $CFL = 0.5$ lies far enough from 1 and is less likely to cause an unexpected instability case. $T = 1$ is long enough for any potential instabilities to come out, while some large value like $T = 10$ may cause phase error.

In order to test the order of convergence and stability, we need both the ℓ_∞ -norm and ℓ_2 -norm of the errors which is the difference between $u_{estimated}$ and u_{exact} at time T . I will run the schemes from $m = 20$ to $m = 10240$ and every iteration multiply 2 to m . Then, let's take the difference of the norms and Δ_x between two iterations.

Based on the definition, we have $E \approx C(\Delta x)^p$, and after applying log, we get $\log(E) \approx \log(C) + p \log(\Delta x)$. According to this, we can get the approximate order of convergence by

$$\frac{\log(diff_norm)}{\log(diff_dx)}$$

For example, the order assigned with the first iteration is 0 while for iteration $i > 0$, the approximate order is calculated by iteration i and $i + 1$.

By the values and trends of the order approximated by ℓ_∞ -norm, we can check them aligning with the theoretical order or not. If we found some potential issue (e.g. the approximate order decreases significantly), we can double check the order approximated by ℓ_2 -norm and two norms. Then, we may conclude the scheme becomes unstable when m increases.

2 Code

```
[71]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import sbp
import scipy.sparse as sp

atol=1e-10

def g(t, c, a):
    return np.exp(-np.pow(a - c * t, 2)) + np.cos(13 * (a - c * t))

def f(x):
    return np.exp(-np.pow(x, 2)) + np.cos(13 * x)

def get_u_t(u, t, c, a, D, Hinv):
    du_dt = - c * D.dot(u)
```

```

e0 = np.zeros(np.shape(u))
e0[0] = 1
penalty = - c * Hinv @ e0 * (u[0] - g(t, c, a))
du_dt += penalty
return du_dt

def run_rk6(sbp_func, m, CFL=0.5, T=1, c=1.1, a=-1, b=1):
    dx = (b - a) / (m - 1)
    xs = np.linspace(a, b, m)
    dt = CFL * dx / abs(c)
    nt = int(np.ceil(T / dt))
    dt = T / nt

    u = f(xs)
    D, H, Hinv = sbp_func(m, dx)
    B = np.zeros((m, m))
    B[0, 0] = -1
    B[m - 1, m - 1] = 1

    LHS = H @ D + D.T.conj() @ H
    hold = np.linalg.norm(LHS.toarray() - B) < atol
    k = 0
    while(np.linalg.norm(D.dot(xs**k) - k * xs**(k - 1)) < atol):
        k += 1
    sbp_order = k

    for j in range(nt):
        k1 = get_u_t(u, j * dt, c, a, D, Hinv)
        k2 = get_u_t(u + dt * k1 / 6, j * dt + dt / 6, c, a, D, Hinv)
        k3 = get_u_t(u + dt * (k1 / 12 + k2 / 12), j * dt + dt / 6, c, a, D,
        Hinv)
        k4 = get_u_t(u + dt * (- 4 * k2 / 33 + 5 * k3 / 11), j * dt + dt / 3,
        c, a, D, Hinv)
        k5 = get_u_t(u + dt * (- k1 / 4 - 29 * k2 / 44 + 31 * k3 / 22), j * dt
        + dt / 2, c, a, D, Hinv)
        k6 = get_u_t(u + dt * (3 * k1 / 11 + 8 * k2 / 33 - 4 * k3 / 11 + k4 /
        11 + 14 * k5 / 33), j * dt + 2 * dt / 3, c, a, D, Hinv)
        k7 = get_u_t(u + dt * (- 17 * k1 / 48 - 5 * k2 / 12 + k3 + k4 - 13 * k5
        / 12 + 11 * k6 / 16), j * dt + 5 * dt / 6, c, a, D, Hinv)
        k8 = get_u_t(u + dt * (20 * k1 / 39 + 12 * k2 / 39 - 31 * k3 / 39 - k4 /
        39 + 34 * k5 / 39 - 11 * k6 / 39 + 16 * k7 / 39), j * dt + dt, c, a, D,
        Hinv)
        u = u + dt * (13 * k1 / 200 + 4 * k3 / 25 + 11 * k4 / 40 + 11 * k6 / 40
        + 4 * k7 / 25 + 13 * k8 / 200)

    u_exact = f(xs - c * T)

```

```

inf_error = np.max(np.abs(u - u_exact))
sq_error = np.sqrt(np.sum((u - u_exact)**2) * dx)
return xs, u, u_exact, dx, inf_error, sq_error, hold, sbp_order

```

3 Test

3.1 SBP(2, 1)

```

[72]: m = 20
dxs, inf_errors, sq_errors, holds, sbp_orders, ms = [], [], [], [], [], []
for i in range(10):
    _, _, u_exact, dx, inf_err, sq_err, hold, sbp_order = run_rk6(sbp.sbp21, m)
    dxs.append(dx)
    inf_errors.append(inf_err)
    sq_errors.append(sq_err)
    holds.append(hold)
    sbp_orders.append(sbp_order)
    ms.append(m)
    m *= 2

inf_orders, sq_orders = [0], [0]
inf_orders.extend(np.diff(np.log(inf_errors)) / np.diff(np.log(dxs)))
sq_orders.extend(np.diff(np.log(sq_errors)) / np.diff(np.log(dxs)))
df = pd.DataFrame([ms, holds, sbp_orders, inf_errors, inf_orders, sq_errors,
    ↪sq_orders]).transpose()
df.columns = ['m', '(1) holds', '(2) Order D', 'linf error', 'linf Order conv',
    ↪'l2 error', 'l2 Order conv']
print(df)

```

	m	(1) holds	(2) Order D	linf error	linf Order conv	l2 error	l2 Order conv
0	20	True	2	1.84e+00	0	1.61e+00	0
1	40	True	2	1.10e+00	7.23e-01	8.71e-01	8.58e-01
2	80	True	2	2.78e-01	1.95e+00	2.13e-01	1.99e+00
3	160	True	2	6.82e-02	2.01e+00	5.21e-02	2.02e+00
4	320	True	2	1.68e-02	2.01e+00	1.29e-02	2.01e+00
5	640	True	2	4.20e-03	2.00e+00	3.20e-03	2.00e+00
6	1280	True	2	1.05e-03	2.00e+00	7.99e-04	2.00e+00
7	2560	True	2	2.61e-04	2.00e+00	1.99e-04	2.00e+00
8	5120	True	2	6.52e-05	2.00e+00	4.98e-05	2.00e+00
9	10240	True	2	1.63e-05	2.00e+00	1.25e-05	2.00e+00

3.2 SBP(4, 2)

```

[73]: m = 20
dxs, inf_errors, sq_errors, holds, sbp_orders, ms = [], [], [], [], [], []
for i in range(10):
    _, _, u_exact, dx, inf_err, sq_err, hold, sbp_order = run_rk6(sbp.sbp42, m)

```

```

dxs.append(dx)
inf_errors.append(inf_err)
sq_errors.append(sq_err)
holds.append(hold)
sbp_orders.append(sbp_order)
ms.append(m)
m *= 2

inf_orders, sq_orders = [0], [0]
inf_orders.extend(np.diff(np.log(inf_errors)) / np.diff(np.log(dxs)))
sq_orders.extend(np.diff(np.log(sq_errors)) / np.diff(np.log(dxs)))
df = pd.DataFrame([ms, holds, sbp_orders, inf_errors, inf_orders, sq_errors,
    ↪sq_orders]).transpose()
df.columns = ['m', '(1) holds', '(2) Order D', 'linf error', 'linf Order conv',
    ↪'l2 error', 'l2 Order conv']
print(df)

```

	m	(1) holds	(2) Order D	linf error	linf Order conv	l2 error	l2 Order conv
0	20	True	3	1.75e+00	0	1.17e+00	0
1	40	True	3	1.50e-01	3.42e+00	1.03e-01	3.38e+00
2	80	True	3	1.31e-02	3.46e+00	9.03e-03	3.45e+00
3	160	True	3	1.29e-03	3.31e+00	9.66e-04	3.20e+00
4	320	True	3	1.53e-04	3.06e+00	1.14e-04	3.07e+00
5	640	True	3	2.04e-05	2.90e+00	1.40e-05	3.02e+00
6	1280	True	3	2.62e-06	2.95e+00	1.74e-06	3.01e+00
7	2560	True	3	3.32e-07	2.98e+00	2.17e-07	3.00e+00
8	5120	True	3	4.18e-08	2.99e+00	2.71e-08	3.00e+00
9	10240	True	3	5.24e-09	2.99e+00	3.38e-09	3.00e+00

3.3 SBP(6, 3)

```

[74]: m = 20
dxs, inf_errors, sq_errors, holds, sbp_orders, ms = [], [], [], [], [], []
for i in range(10):
    _, _, u_exact, dx, inf_err, sq_err, hold, sbp_order = run_rk6(sbp.sbp63, m)
    dxs.append(dx)
    inf_errors.append(inf_err)
    sq_errors.append(sq_err)
    holds.append(hold)
    sbp_orders.append(sbp_order)
    ms.append(m)
    m *= 2

inf_orders, sq_orders = [0], [0]
inf_orders.extend(np.diff(np.log(inf_errors)) / np.diff(np.log(dxs)))
sq_orders.extend(np.diff(np.log(sq_errors)) / np.diff(np.log(dxs)))

```

```

df = pd.DataFrame([ms, holds, sbp_orders, inf_errors, inf_orders, sq_errors,
    ↪sq_orders]).transpose()
df.columns = ['m', '(1) holds', '(2) Order D', 'linf error', 'linf Order conv',
    ↪'l2 error', 'l2 Order conv']
print(df)

```

	m	(1) holds	(2) Order D	linf error	linf Order conv	l2 error	l2 Order conv
0	20	True	4	1.04e+00	0	8.13e-01	0
1	40	True	4	9.47e-02	3.33e+00	7.30e-02	3.35e+00
2	80	True	4	9.27e-03	3.29e+00	5.15e-03	3.76e+00
3	160	True	4	1.11e-03	3.03e+00	3.56e-04	3.82e+00
4	320	True	4	7.75e-05	3.82e+00	2.26e-05	3.96e+00
5	640	True	4	2.31e-06	5.06e+00	1.42e-06	3.98e+00
6	1280	True	4	2.79e-07	3.05e+00	9.22e-08	3.94e+00
7	2560	True	4	3.30e-08	3.08e+00	5.87e-09	3.97e+00
8	5120	True	4	2.55e-09	3.69e+00	3.66e-10	4.00e+00
9	10240	True	0	1.78e-10	3.84e+00	2.28e-11	4.01e+00

3.4 SBP(4, 4)

```

[75]: m = 20
dxs, inf_errors, sq_errors, holds, sbp_orders, ms = [], [], [], [], [], []
for i in range(10):
    _, _, u_exact, dx, inf_err, sq_err, hold, sbp_order = run_rk6(sbp.sbp44, m)
    dxs.append(dx)
    inf_errors.append(inf_err)
    sq_errors.append(sq_err)
    holds.append(hold)
    sbp_orders.append(sbp_order)
    ms.append(m)
    m *= 2

inf_orders, sq_orders = [0], [0]
inf_orders.extend(np.diff(np.log(inf_errors)) / np.diff(np.log(dxs)))
sq_orders.extend(np.diff(np.log(sq_errors)) / np.diff(np.log(dxs)))
df = pd.DataFrame([ms, holds, sbp_orders, inf_errors, inf_orders, sq_errors,
    ↪sq_orders]).transpose()
df.columns = ['m', '(1) holds', '(2) Order D', 'linf error', 'linf Order conv',
    ↪'l2 error', 'l2 Order conv']
print(df)

```

	m	(1) holds	(2) Order D	linf error	linf Order conv	l2 error	l2 Order conv
0	20	True	4	1.59e+00	0	1.15e+00	0
1	40	True	4	1.15e-01	3.66e+00	7.88e-02	3.73e+00
2	80	True	4	7.16e-03	3.93e+00	4.82e-03	3.96e+00
3	160	True	4	4.35e-04	4.00e+00	2.96e-04	3.99e+00
4	320	True	4	2.69e-05	4.00e+00	1.83e-05	4.00e+00
5	640	True	4	1.68e-06	3.99e+00	1.14e-06	4.00e+00

6	1280	True	4	1.04e-07	4.01e+00	7.09e-08	4.00e+00
7	2560	True	4	6.49e-09	4.00e+00	4.42e-09	4.00e+00
8	5120	True	4	4.06e-10	4.00e+00	2.76e-10	4.00e+00
9	10240	True	5	2.54e-11	4.00e+00	1.73e-11	4.00e+00

4 Discussion

Operator	Theoretical Order of D	Actual Order of D (l2)	Theoretical Order of Conv	Actual Order of Conv (linf)	Stable vs. Unstable
SBP(2, 1)	1	2	1	2	Stable
SBP(4, 2)	2	3	2	3	Stable
SBP(6, 3)	3	4 (unstable if m is large)	3	4	Stable
SBP(4, 4)	4	4 (unstable if m is large)	4	4	Stable

First, we verify that the equation (1) holds for all of our SBP with different spatial steps m . Then, We are able to use them as SBP operators.

We expect the order of operator D of SBP(a , b) to be $\min\{a, b\}$ theoretically. Furthermore, the order of convergence is $\min\{\text{order of } D, \text{order of scheme}\}$ theoretically.

However, the actual order of D we get from l_2 -norm is 1 bigger than expected for SBP(2, 1), SBP(4, 2), SBP(6, 3). The possible reason is due to the symmetries of the three operators. Also, we observe that the orders of both SBP(6, 3) and SBP(4, 4) become unstable when m becomes large (e.g. 10240). They first increase a little and then drop to 0 as m increases. The possible reason is when we separate the interval too much, each distance is too small and may cause underflow during computation. Hence, we'd better to limit m smaller than 5000 for best performance.

Since we use RK6 scheme (order 6, bigger than the order of any operator), the order of convergence should be same as the order of D . By observation (l_∞ -norm), the actual order of conv is equal to the actual order of D which is consistent. For SBP(6, 3), the order from l_∞ -norm will fluctuate at first, but same as other operators, it will be stable around 4 at last. Also, the order from l_2 -norm for each of the operator is approaching their order of D gradually as wanted. Therefore, we can conclude RK6 with all four SBP operators are stable.

Comparing with ghost cells or one-sided difference formula we used to treat boundary nodes in project 2, we can get stable and higher order of convergence by designing corresponding SBP operators. For RK4C4 with 4 ghost cells, the theoretical order of convergence should be 4 but our actual order drops from 4 when the number of points increases and is around 2 at last, which is unstable. The reason is because the order 2 on the boundary points affect the order on the whole space. However, our SBP-SAT methods don't have this problem. Based on tests, our RK6 with SBP operators are stable. We can design SBP(a , b) with high order as long as the order is smaller than

or equal to RK6. Therefore, SBP-SAT method is a way to get stable and high order convergence scheme for finite difference methods.